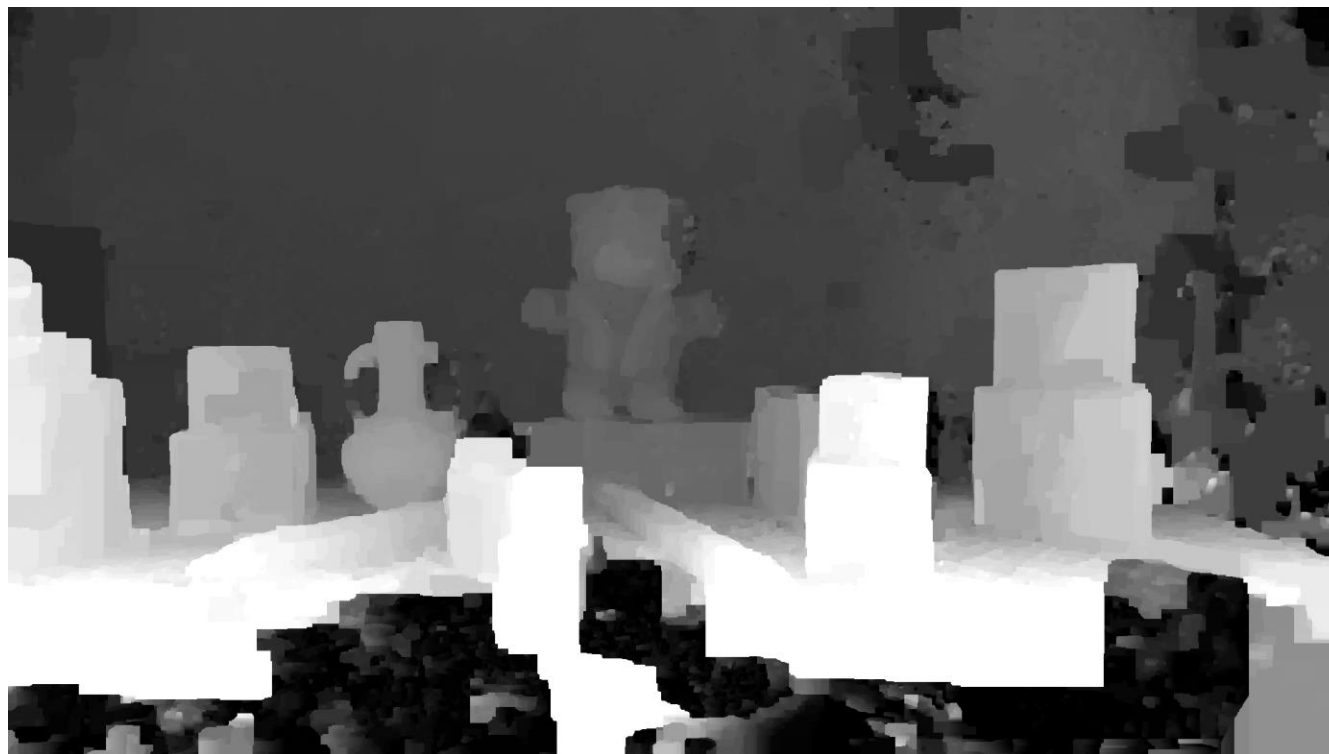
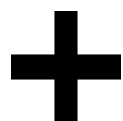


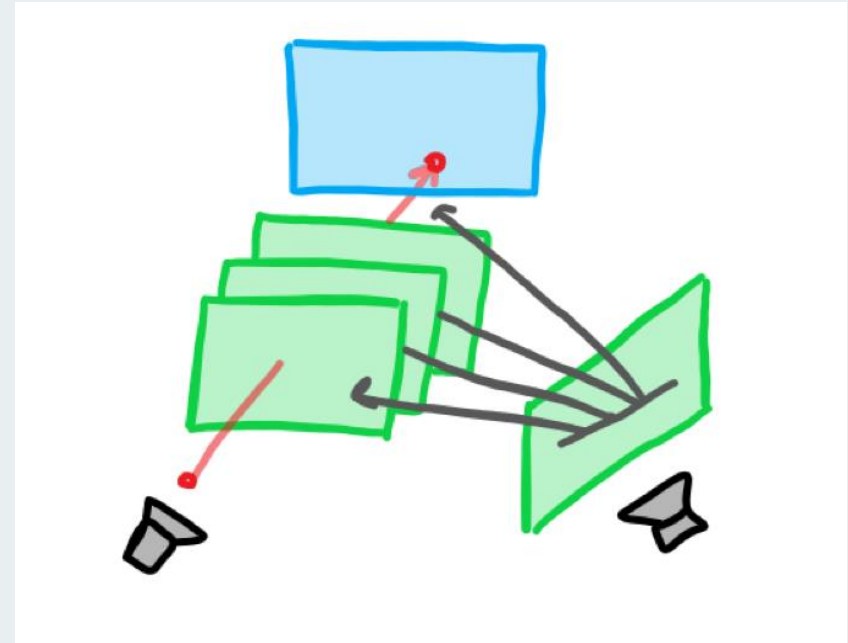
Planesweep

Depth Estimation using
multiple cameras



Quick Recap

- Estimating the **depth** of a scene can be **useful** in a lot of different contexts (i.e. robotics, ML, ...).
- The **Planesweep algorithm** allows for a pretty good estimation!
- It uses **point projection** and **pixel similarity** to estimate depth.
- It is **highly** possible to **parallelize**.



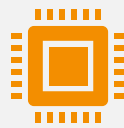
We will use the **CUDA framework** to make it into a fully fledged **parallel version** that make use of our many **cores** and **threads** (I've only got a GTX 1060 but that will do).



Using the power of CUDA!



NVIDIA
CUDA[®]



Instead of computing the cost cube in a sequential way, we will be doing it all at the same time!



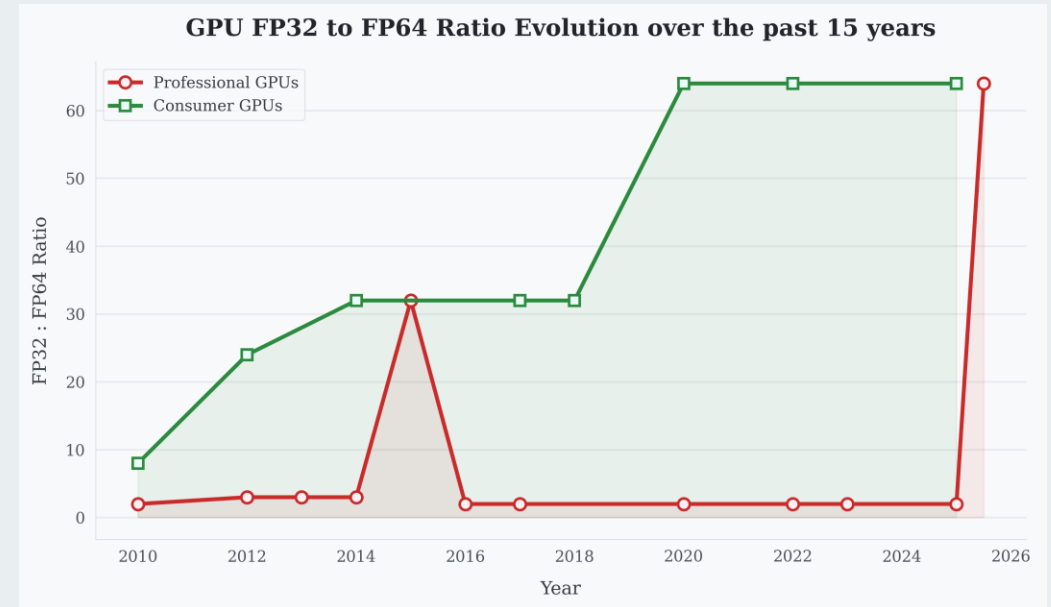
Each thread will be assigned to a tuple (x, y, z) , where z is a potential plane. We use 256 planes.



We can do that since the cost cube doesn't depend on other data to be constructed. It is why the planesweep algorithm is highly parallelizable.

Data types

- In the original implementation, we used **complex types** such as **OpenCV Matrices**, **std::vectors** or **cams**.
- We replace them with simple structures such as **contiguous arrays**.
- We also use floats instead of doubles. It has to do with the proportion of **FP32** units in my GPU, in comparaison of **FP64** units.



- The tradeoff is precision.
- In our case, we do not care about it, since it's *barely* noticeable.

Source: <https://nicolasdickenmann.com/blog/the-great-fp64-divide.html>



LDG Cache



- We use **LDG** (Load Global) **cache**. It allows for read-only cache.
- Since the cam images will only **be read** and **never modified**, it is clearly a better approach to increase memory efficiency.
- We avoid accessing global memory.
- In **CUDA**, method is called **__ldg()**.

CUDA Memory Types: Know Where Your Data Lives

Registers PER THREAD	~1 cycle ~255 regs/thread Thread only Fastest. Local variables go here automatically. Private to each thread — nothing to declare.
Shared Memory PER BLOCK	~5 cycles 48-96 KB/SM All threads in block On-chip, user-managed. Declare with __shared__ . Load once, reuse many times. Always follow with __syncthreads() .
Global Memory ALL THREADS	~500 cycles GBs (VRAM) All threads + host Main GPU DRAM. Where cudaMalloc allocates. Largest but slowest — every optimization is about reducing hits here.
Local Memory PER THREAD	~500 cycles per thread Thread only Spills from registers to DRAM when a thread uses too many variables. Same latency as global memory — avoid register spilling.
Constant Memory READ-ONLY	~5 cycles (cached) 64 KB All threads Read-only, cached on-chip. Fast when all threads in a warp read the same address. Ideal for weights, config, lookup tables.

Rule: push data as close to the cores as possible → Registers → Shared → Constant → Global

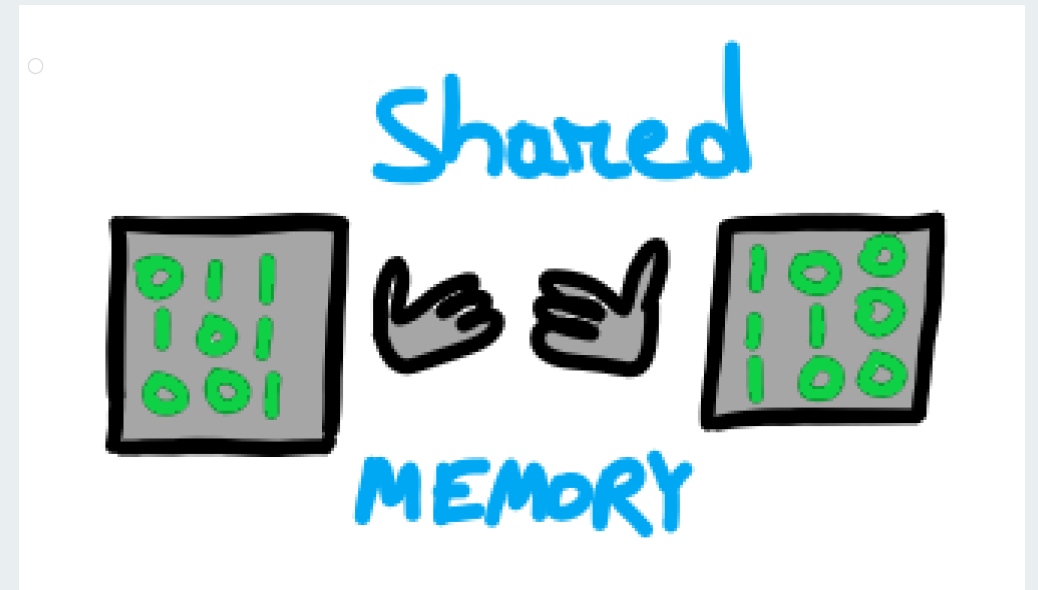
Latency values approximate — vary by GPU architecture

Source: https://www.linkedin.com/posts/pranali-desai_cuda-gpu-memoryoptimization-activity-7431418413740105728-h9SS/



Shared Memory

- We also use **shared memory** to reduce global memory accesses during the computation of the **cost cube**.
- The ref image is loaded into a **tile of shared memory**. Each block allocates a region that includes the pixels required for the **matching window**.
- Loading is performed **cooperatively**, each thread does some work and then we call `__syncthread()` to ensure data is properly loaded in the shared memory.



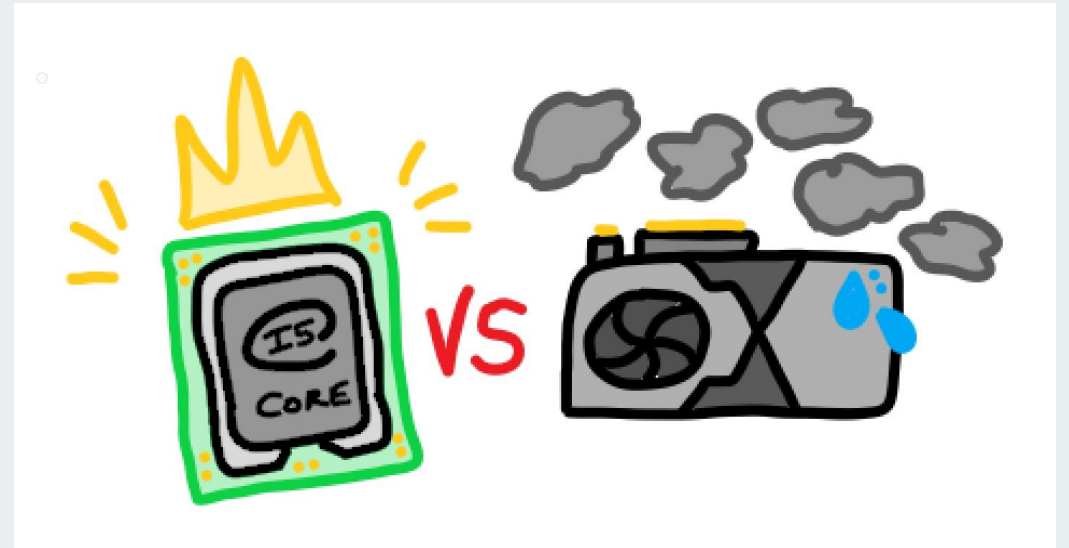
Work per Block

- Each **CUDA block** is responsible for computing **multiple** depth planes (**2** in my implementation).
- It allows to reuse to same shared memory for multiple depth hypothesis.
- Trough testing, 2 planes per block was a **good fit**.
- About **block dim**, we use **<<<32, 8>>>** so make use of our **warp size**. It scales nicely to fit our 256 planes.



Experimentation

- **NVIDIA GTX 1060** under **WSL** environment on **Ubuntu 24.04.2 LTS**.
- **AMD Ryzen 5 9600X** on **Windows 10**.
- Full specs can be found in the report.
- My CPU is way better than my GPU if we had to compare, so speedup could potentially scale even more, about **2~10x**.



Results

- Over **10 iterations**, full details can also be found in the report.
- We dropped **first iteration** to mitigate "**slow start**" effect, present on my GTX1060.

Implementation	PlaneSweep time	Argmin time	Total time	Speedup
Naïve CPU	442 s	1681 ms	455 s	1×
Naïve GPU	4361 ms	1683 ms	6 s	75.8×
Optimized GPU	722 ms	12 ms	1.909 s	238.4×

Table 1: Performance comparison of implementations of Plane Sweep

Further improvements

- Better use of shared memory.
- Improve memory coalescing.



Bibliography

- Richard Szeliski. Image Formation. Computer Vision Algorithms and Application, Springer, chap. 2, 2011.
- Richard Szeliski. Stereo Correspondance. Computer Vision Algorithms and Application, Springer, chap. 11, 2011.
- My GitHub (You can find the implementation): <https://github.com/Ant0in>